

PROMPT_SYSTEM.md

Version: 0.1

Status: Draft

1. Purpose

This document defines how SpeechStudio constructs Higgs prompts.

It describes how emotions, styles, prosody, sound effects and reference voice data are converted into the final input text sent to the model.

The user should not manually manage tokens in normal mode.

The application must generate valid prompts automatically.

2. Prompt Construction Principle

SpeechStudio builds a single final prompt from user selected controls and plain text.

The prompt may contain:

emotion tokens

style tokens

prosody tokens

sound effect tokens

reference voice context

plain user text

Developer mode may show the raw prompt.

Normal mode should hide token syntax from the user as much as possible.

3. Supported Token Categories

SpeechStudio must support all token categories currently documented by Higgs Audio V3.

Emotion

Style

Prosody

Sound Effects

Reference Context

Each category has its own rules.

4. Emotion Tokens

Emotion tokens are sentence level tokens.

They must be placed at the start of the sentence or prompt block.

Only one emotion should be active by default for one sentence block.

Supported emotions

Elation

Amusement

Enthusiasm

Determination

Pride

Contentment

Affection

Relief

Contemplation

Confusion

Surprise

Awe

Longing

Arousal

Anger

Fear

Disgust

Bitterness

Sadness

Shame

Helplessness

Example

```
``text id="p1" <|emotion:awe|>Az univerzum tele van titkokkal.
```

Example

```
```text id="p2"  
<|emotion:elation|>Ez fantasztikus! Végre sikerült!
```

---

## 5. Style Tokens

Style tokens are sentence level tokens.

They must be placed at the start of the sentence or block.

Supported styles

Singing

Shouting

Whispering

## Example

```
```text id="p3" <|style:whispering|>Most nagyon halkán beszélek.
```

Example

```
```text id="p4"
<|style:singing|>Tavaszi szél vizet áraszt.
```

## Example

```
```text id="p5" <|style:shouting|>Most azonnal figyelj rám!
```

6. Prosody Tokens

Prosody tokens are split into sentence level and inline usage.

6.1 Sentence level prosody

Speed

Very Slow

Slow

Normal

Fast

Very Fast

Pitch

Low

Normal

High

Delivery

Expressive High

Expressive Low

Examples

```
```text id="p6"
<|prosody:speed_fast|>Most gyorsabban beszélek.
```

```
```text id="p7" <|prosody:pitch_low|>Ez mélyebb hangnak hat.
```

```
```text id="p8"
<|prosody:expressive_high|>Ez sokkal élénkebb előadás.
```

## 6.2 Inline prosody

Pause

Long Pause

Examples

```
```text id="p9" Ez egy mondat <|prosody:pause|> és itt jön a folytatás.
```

```
```text id="p10"
Ez egy hosszabb szünet <|prosody:long_pause|> majd utána új rész.
```

---

## 7. Sound Effect Tokens

Sound effect tokens are inline tokens.

The token must be placed immediately before the onomatopoeia.

There must be no space between the token and the suggested sound word.

Supported sound effects

Cough

Laughter

Crying

Screaming

Burping

Humming

Sigh

Sniff

Sneeze

Suggested onomatopoeia

Cough Ahem

Laughter Haha Hehe

Crying Boohoo Sob

Screaming Ahh Aaah

Burping Burp

Humming Hmm Mmm

Sigh Uh Ahh

Sniff Sff

Sneeze Achoo

Examples

```
``text id="p11" <|sfx:laughter|>Haha, ez tényleg működik!
```

```
``text id="p12"
<|sfx:cough|>Ahem, kezdjük újra.
```

```
``text id="p13" <|sfx:sigh|>Ahh, végre sikerült.
```

---

## # 8. Token Placement Rules

Emotion

Sentence start only

Style

Sentence start only

Prosody sentence level

Sentence start only

Prosody inline

Inserted at exact position

Sound effects

Inserted immediately before the onomatopoeia

The application must not place tokens incorrectly.

The application must validate token order before generation.

---

## # 9. Combination Rules

Different token categories may be combined.

Supported combination examples

Emotion plus style

Emotion plus prosody

Emotion plus sfx

Emotion plus style plus prosody

Emotion plus prosody plus sfx

Example

```
```text id="p14"
<|emotion:amusement|><|prosody:expressive_high|>Ez most tényleg vicces.
```

Example

```
```text id="p15" <|emotion:elation|><|sfx:laughter|>Haha, ez fantasztikus!
```

Example

```
```text id="p16"
<|emotion:contemplation|><|prosody:speed_slow|>Érdemes ezt egy pillanatra
átgondolni.
```

The application should allow valid combinations.

The application should warn about conflicting combinations.

10. Conflict Handling

The following conflicts should be handled by the UI and prompt builder.

Multiple emotions in one sentence block

Multiple styles in one sentence block

Conflicting pitch values

Conflicting speed values

Conflicting delivery values

Duplicate SFX tags in the same inline position

The system must warn the user if the prompt becomes ambiguous.

Developer mode may allow manual override.

Normal mode should prefer one active control per category.

11. Prompt Builder Responsibility

The Prompt Builder must be the only component that assembles final Higgs tokens.

The UI must not directly write final token syntax unless Developer Mode is active.

The Prompt Builder receives

selected emotion

selected style

selected prosody

selected sound effect

plain text

reference voice data

generation settings

The Prompt Builder outputs

final prompt string

token preview

validation warnings

12. Prompt Preview

The prompt preview must show the exact model input.

It should display

tokenized prompt

plain text sections

inline sfx positions

reference text when present

The preview should be read only by default.

Developer Mode may enable manual editing.

13. Reference Voice Context

If voice cloning is enabled, the prompt system must include the reference voice context.

This includes

reference audio file

reference transcript

optional metadata

reference token count

The prompt builder must account for reference token insertion.

Reference context is not shown as a normal token block in the user editor.

14. Reference Audio Workflow

The system must support a reference audio based prompt flow.

User selects a voice profile

Voice profile contains reference audio

Voice profile may contain transcript

Prompt builder receives reference context

Model generates output in cloned voice

The prompt system must ensure the reference context is valid before generation.

15. Voice Profile to Prompt Mapping

Every voice profile may influence prompt generation.

Voice profile can provide

reference audio

reference transcript

voice name

description

tags

optional preferred settings

The voice profile is used by the prompt system as a generation context, not as a visible token.

16. Sample Prompt Templates

The application may store reusable templates.

Template example

Podcast

```
```text id="p17" <|emotion:enthusiasm|>Üdvözöllek a mai adásban. Ma egy érdekes témáról beszélünk.
```

Narrator

```
```text id="p18"
<|emotion:awe|><|prosody:speed_slow|>Az univerzum tele van titkokkal.
```

Whisper

```
```text id="p19" <|style:whispering|>Közelebb hajolva most elmondok valamit.
```

Horror

```
```text id="p20"
<|emotion:fear|><|prosody:pitch_low|><|prosody:speed_slow|>Valami mozog
odakint.
```

17. Token Validation

Before generation the prompt system must validate:

unknown token names

invalid category usage

empty prompt

duplicate sentence level controls

invalid SFX placement

conflicting prosody tokens

missing reference transcript when required

invalid text encoding

If validation fails, generation must not start.

18. Token Preview Modes

Basic mode

Shows human readable labels only

Advanced mode

Shows labels plus token syntax

Developer mode

Shows exact final prompt text and internal parsing diagnostics

19. Supported GUI Mapping

Each control in the GUI must map to a known prompt token or generation parameter.

Emotion selector

Emotion token

Style selector

Style token

Speed selector

Speed token

Pitch selector

Pitch token

Delivery selector

Delivery token

SFX selector

SFX token plus onomatopoeia

Text editor

Plain user text

Voice selector

Reference voice context

Reference transcript editor

Reference text

The prompt system must keep this mapping deterministic.

20. Prompt Serialization

The system must be able to serialize prompts for history and export.

Saved prompt data should include

raw user text

inserted tokens

final prompt string

selected voice

generation settings

timestamp

This allows exact reproduction of a generation later.

21. History Integration

Every generated prompt must be stored together with its settings.

The prompt system is responsible for exporting the prompt portion of history.

History entries should allow reuse of the exact same prompt.

22. Preset Integration

Presets may store prompt related selections.

Emotion

Style

Prosody

SFX

Voice profile

Seed

Sampling parameters

Prompt template

The prompt system must be able to rebuild a prompt from a preset without manual intervention.

23. Example Full Prompts

Example 1

```
```text id="p21" <|emotion:elation|><|prosody:expressive_high|>Ez fantasztikus! Végre működik minden.
```

Example 2

```
```text id="p22"
<|style:whispering|><|emotion:contemplation|>Most halkán beszélek, mert ezt
jobban át kell gondolni.
```

Example 3

```
```text id="p23" <|emotion:amusement|><|sfx:laughter|>Haha, ezt nem hiszem el!
```

Example 4

```
```text id="p24"
<|emotion:fear|><|prosody:pitch_low|><|prosody:speed_slow|>Valaki jár a
folyosón.
```

24. Output Requirements

The final prompt must be:

syntactically valid

token order correct

human readable in preview

compatible with Higgs

serializable to history

reconstructable from saved settings

25. Non Goals

The prompt system does not decide voice quality.

The prompt system does not perform audio decoding.

The prompt system does not manage CUDA.

The prompt system does not download models.

The prompt system only assembles and validates prompt content.

26. Success Criteria

The prompt system is successful if the user can select controls in the GUI and the application produces a correct Higgs prompt without requiring manual token editing.